MO - Erick

Aplicação do algoritmo de MO. Imprime quantos elementos tem frequência maior que 1 em um dado intervalo. Complexidade: O((N+M)*log(N) + N*log(N))

```
#include<bits/stdc++.h>
#define MAXN 100000
#define MAXM 100000

using namespace std;

struct query
{
   int L, R, A, ind, BL;
};

int total;
int ans[MAXM+1], freq[MAXN+1], B[MAXN+1];

query queries[MAXM+1];

bool cmp(query x, query y)
{
   if(x.BL != y.BL)
      return x.BL < y.BL;
   return x.R < y.R;
}

void checkmais(int x)
{
   freq[B[x]]++;
   if(freq[B[x]] == 2)
      total += 2;
   if(freq[B[x]] > 2)
      total++;
}

void checkmenos(int x)
{
   if(x == 0)
      return;
   freq[B[x]]--;
   if(freq[B[x]] > 1)
      total--;
   if(freq[B[x]] == 1)
      total -= 2;
}

void MO(int M)
{
   int i, L, R, curL, curR;
   sort(queries, queries+M, cmp);
   curL = queries[0].L;
   curR = queries[0].R;
   for(int i = curL; i <= curR; i++)
      checkmais(i);
   ans[queries[0].ind] = total;
   for(i = 1; i < M; i++)
   {
      L = queries[i].L;
      R = queries[i].R;
      while(curL < L)
         checkmenos(curL++);
      while(curL > L)
         checkmais(--curL);
      while(curR < R)
         checkmais(++curR);
      while(curR > R)
         checkmenos(curR--);
      ans[queries[i].ind] = total;
   }
}

int main()
{
   int N, M, block;
   scanf("%d", &N);
   for(int i = 1; i <= N; i++)
      scanf("%d", &B[i]);
   scanf("%d", &M);
   block = sqrt(N);
   for(int i = 0; i < M; i++)
   {
      scanf("%d %d", &queries[i].L, &
          queries[i].R);
      queries[i].ind = i;
      queries[i].BL = queries[i].L/block;
   }
   MO(M);
   for(int i = 0; i < M; i++)
      printf("%d\n", ans[i]);
   return 0;
}
```

Min queue

Dado array a e um tamanho de janela k, imprime os menores elementos de toda janela de tamanho k da esquerda para a direita em O(N)

```
#include<bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0);
#define MAXN 100021
#define ll long long
#define ii pair<ll, ll>
#define fs first
#define sc second
#define vi vector<ll>

using namespace std;

int n, k;
ll a[MAXN];
deque<ii> dq;
vi ans;

int main(){_
   while((cin >> n >> k) && (n || k)){
      for(int i = 0; i < n; ++i){
         cin >> a[i];
      }

      ans.clear();
      dq.clear();
      dq.push_back(ii(a[0], 0));

      for(int i = 1; i < n; ++i){
         if(a[i] <= dq.front().fs) {
            dq.clear();
            dq.push_back(ii(a[i], i));
         }
         else {
            while(dq.back().fs >= a[i])
               dq.pop_back();
            dq.push_back(ii(a[i], i));
         }

         if((i + 1) >= k){
            while((i - dq.front().sc + 1)
                  > k) dq.pop_front();
            ans.push_back(dq.front().fs);
         }
      }

      int lim = n - k + 1;
      cout << ans[0];
      for(int i = 1; i < lim; ++i) cout
          << ' ' << ans[i];
      cout << endl;
   }

   return 0;
}
```

MO - geeks for geeks

```
// Program to compute sum of ranges for
    different range
// queries
#include <bits/stdc++.h>
using namespace std;

// Variable to represent block size. This
    is made global
// so compare() of sort can use it.
int block;

// Structure to represent a query range
struct Query
{
    int L, R;
};

// Function used to sort all queries so
    that all queries
// of the same block are arranged
    together and within a block,
// queries are sorted in increasing order
    of R values.
bool compare(Query x, Query y)
{
    // Different blocks, sort by block.
    if (x.L/block != y.L/block)
        return x.L/block < y.L/block;

    // Same block, sort by R value
    return x.R < y.R;
}

// Prints sum of all query ranges. m is
    number of queries
// n is size of array a[].
void queryResults(int a[], int n, Query q
    [], int m)
{
    // Find block size
    block = (int)sqrt(n);

    // Sort all queries so that queries
        of same blocks
    // are arranged together.
    sort(q, q + m, compare);

    // Initialize current L, current R
        and current sum
    int currL = 0, currR = 0;
    int currSum = 0;

    // Traverse through all queries
    for (int i=0; i<m; i++)
    {
        // L and R values of current
            range
        int L = q[i].L, R = q[i].R;

        // Remove extra elements of
            previous range. For
        // example if previous range is
            [0, 3] and current
        // range is [2, 5], then a[0] and
            a[1] are subtracted
        while (currL < L)
        {
            currSum -= a[currL];
            currL++;
        }

        // Add Elements of current Range
        while (currL > L)
        {
            currSum += a[currL-1];
            currL--;
        }
        while (currR <= R)
        {
            currSum += a[currR];
            currR++;
        }

        // Remove elements of previous
            range.  For example
        // when previous range is [0, 10]
            and current range
        // is [3, 8], then a[9] and a[10]
            are subtracted
        while (currR > R+1)
        {
            currSum -= a[currR-1];
            currR--;
        }

        // Print sum of current range
        cout << "Sum of [" << L << ", "
            << R
             << "] is " << currSum <<
                endl;
    }
}

// Driver program
int main()
{
    int a[] = {1, 1, 2, 1, 3, 4, 5, 2,
        8};
    int n = sizeof(a)/sizeof(a[0]);
    Query q[] = {{0, 4}, {1, 3}, {2, 4}};
    int m = sizeof(q)/sizeof(q[0]);
    queryResults(a, n, q, m);
    return 0;
}
```

Caminhos ótimos simétricos - Quantidade de caminhos mínimos

Calcula a quantidade de caminhos mínimos de (0,0) e (n-1,n-1) até a diagonal principal - Dijkstra

```
#include<bits/stdc++.h>
#define MAXN 123
#define vi vector<int>
#define ii pair<int, int>
#define pp pair<ii, ii>
#define ipp pair<int, pp>
#define vipp vector<ipp>
#define bs bitset<11133>
#define ib pair<ii, bs>

using namespace std;

    // cima, direita, baixo, esquerda -
        respectivamente
int diu[] = {-1, 0, 1, 0}, dju[] = {0, 1,
     0, -1};
    // direita, cima, esquerda, baixo -
        respectivamente
int did[] = {0, -1, 0, 1}, djd[] = {1, 0,
     -1, 0};
int mat[MAXN][MAXN], ans;
map<pp, int> dist;
int n;
ll qtd[MAXN][MAXN];
int pode(int i, int j)
    return ((i > 0) && (j > 0) && (i <= n)
        && (j <= n));
int vai(int i, int j)
    return (i * n) + j;
int djk(){
    priority_queue<ipp, vipp, greater<ipp>
        > pq;
    for(int i = 1; i <= n; ++i){
        for(int j = 1; j <= n; ++j){
            if(i + j <= n + 1){
                dist[pp(ii(i, j), ii(n + 1 -
                    j, n + 1 - i))] = INF;
            }
            qtd[i][j] = 1;
        }
    }
    pp s = pp(ii(1, 1), ii(n, n));
    dist[s] = mat[1][1] + mat[n][n];
    pq.push(ipp(dist[s], s));
    while(!pq.empty()){
        ipp topo = pq.top(); pq.pop();
        int d = topo.fs;
        pp u = topo.sc;
        if(d > dist[u]) continue;
//      if(u.fs.fs + u.fs.sc == n + 1)
     return dist[u] - mat[u.fs.fs][u.fs.sc
     ];
        for(int i = 0; i < 4; ++i){
            if(pode(u.fs.fs + diu[i], u.fs.
                sc + dju[i]) && pode(u.sc.fs
                 + did[i], u.sc.sc + djd[i])
                 ){
                pp v = pp(ii(u.fs.fs + diu[i
                    ], u.fs.sc + dju[i]), ii(
                    u.sc.fs + did[i], u.sc.sc
                     + djd[i]));
                int w = mat[v.fs.fs][v.fs.sc]
                     + mat[v.sc.fs][v.sc.sc];
                if(dist[u] + w == dist[v])
                    qtd[v.fs.fs][v.fs.sc] = (
                    qtd[v.fs.fs][v.fs.sc] +
                    qtd[u.fs.fs][u.fs.sc])%
                    mod;
                if(dist[u] + w < dist[v]){
                   qtd[v.fs.fs][v.fs.sc] =
                       qtd[u.fs.fs][u.fs.sc];
                   dist[v] = dist[u] + w;
                   pq.push(ipp(dist[v], v));
                }
            }
        }
    }
    int ret = 0, menor = INF, j;
    ll ans = 0;
    pp p;
    for(int i = 1; i <= n; ++i){
        j = n + 1 - i;
        p.fs.fs = i; p.fs.sc = j;
        p.sc.fs = i; p.sc.sc = j;
        dist[p] -= mat[i][j];
        if(dist[p] < menor){
            menor = dist[p];
        }
    }
    for(int i = 1; i <= n; ++i){
        j = n + 1 - i;
        p.fs.fs = i; p.fs.sc = j;
        p.sc.fs = i; p.sc.sc = j;
        if(dist[p] == menor){
            ans = (ans + qtd[i][j])%mod;
        }
    }
    return ans;
}
int main(){_
    while((cin >> n) && n){
        for(int i = 1; i <= n; ++i){
            for(int j = 1; j <= n; ++j){
                cin >> mat[i][j];
            }
        }
        ans = djk();
        cout << ans << endl;
    }
    return 0;
}
```

Longest Palindromic Substring - O(1) space program

A O(n*n) time and O(1) space program to find the longest palindromic substring

```
#include <stdio.h>
#include <string.h>

// A utility function to print a
    substring str[low..high]
void printSubStr(char* str, int low, int
    high)
{
    for( int i = low; i <= high; ++i )
        printf("%c", str[i]);
}

// This function prints the longest
    palindrome substring (LPS)
// of str[]. It also returns the length
    of the longest palindrome
int longestPalSubstr(char *str)
{
    int maxLength = 1;  // The result (
        length of LPS)

    int start = 0;
    int len = strlen(str);

    int low, high;

    // One by one consider every
        character as center point of
    // even and length palindromes
    for (int i = 1; i < len; ++i)
    {
        // Find the longest even length
            palindrome with center points
        // as i-1 and i.
        low = i - 1;
        high = i;
        while (low >= 0 && high < len &&
            str[low] == str[high])
        {
            if (high - low + 1 >
                maxLength)
            {
                start = low;
                maxLength = high - low +
                    1;
            }
            --low;
            ++high;
        }

        // Find the longest odd length
            palindrome with center
        // point as i
        low = i - 1;
        high = i + 1;
        while (low >= 0 && high < len &&
            str[low] == str[high])
        {
            if (high - low + 1 >
                maxLength)
            {
                start = low;
                maxLength = high - low +
                    1;
            }
            --low;
            ++high;
        }
    }

    printf("Longest palindrome substring
        is: ");
    printSubStr(str, start, start +
        maxLength - 1);

    return maxLength;
}

// Driver program to test above functions
int main()
{
    char str[] = "sdasdannac";
    printf("\nLength is: %d\n",
        longestPalSubstr( str ) );
    return 0;
}
```

Longest Palindromic Substring - O(n*n) space program

O(n*n), mas tem o manachar em O(n), ou seja, não usa isso.

```
#include <stdio.h>
#include <string.h>

// A utility function to print a
    substring str[low..high]
void printSubStr( char* str, int low, int
     high )
{
    for( int i = low; i <= high; ++i )
        printf("%c", str[i]);
}

// This function prints the longest
    palindrome substring
// of str[].
// It also returns the length of the
    longest palindrome
int longestPalSubstr( char *str )
{
    int n = strlen( str ); // get length
        of input string

    // table[i][j] will be false if
        substring str[i..j]
    // is not palindrome.
    // Else table[i][j] will be true
    bool table[n][n];
    memset(table, 0, sizeof(table));

    // All substrings of length 1 are
        palindromes
    int maxLength = 1;
    for (int i = 0; i < n; ++i)
        table[i][i] = true;

    // check for sub-string of length 2.
    int start = 0;
    for (int i = 0; i < n-1; ++i)
    {
        if (str[i] == str[i+1])
        {
            table[i][i+1] = true;
            start = i;
            maxLength = 2;
        }
    }

    // Check for lengths greater than 2.
        k is length
    // of substring
    for (int k = 3; k <= n; ++k)
    {
        // Fix the starting index
        for (int i = 0; i < n-k+1 ; ++i)
        {
            // Get the ending index of
                substring from
            // starting index i and
                length k
            int j = i + k - 1;

            // checking for sub-string
                from ith index to
            // jth index iff str[i+1] to
                str[j-1] is a
            // palindrome
            if (table[i+1][j-1] && str[i]
                 == str[j])
            {
                table[i][j] = true;

                if (k > maxLength)
                {
                    start = i;
                    maxLength = k;
                }
            }
        }
    }

    printf("Longest palindrome substring
        is: ");
    printSubStr( str, start, start +
        maxLength - 1 );

    return maxLength; // return length of
         LPS
}

// Driver program to test above functions
int main()
{
    char str[] = "forgeeksskeegfor";
    printf("\nLength is: %d\n",
        longestPalSubstr( str ) );
    return 0;
}
```

Longest Palindromic Subsequence

Imprime o tamanho da maior subsequencia (sequência não contigua) palindromo. Complexidade O(N*N)

```
#include<bits/stdc++.h>

int lps(char *seq, int i, int j) {
   // Base Case 1: If there is only 1
       character
   if (i == j)
     return 1;

   // Base Case 2: If there are only 2
       characters and both are same
   if (seq[i] == seq[j] && i + 1 == j)
     return 2;

   // If the first and last characters
       match
   if (seq[i] == seq[j])
      return lps (seq, i+1, j-1) + 2;

   // If the first and last characters do
       not match
   return max( lps(seq, i, j-1), lps(seq,
       i+1, j) );
}

int main() {
    char seq[] = "GEEKSFORGEEKS";
    int n = strlen(seq);
    printf ("The length of the LPS is %d\
        n", lps(seq, 0, n-1));
    return 0;
}
```

Print Longest Palindromic Subsequence

Print longest palindromic subsequence. Complexidade: O(N*N)

```
#include<bits/stdc++.h>
using namespace std;
string lcs(string &X, string &Y) {
    int m = X.length();
    int n = Y.length();
    int L[m+1][n+1];

    /* Following steps build L[m+1][n+1]
        in bottom
       up fashion. Note that L[i][j]
           contains
       length of LCS of X[0..i-1] and Y
           [0..j-1] */
    for (int i=0; i<=m; i++) {
        for (int j=0; j<=n; j++) {
            if (i == 0 || j == 0)
                L[i][j] = 0;
            else if (X[i-1] == Y[j-1])
                L[i][j] = L[i-1][j-1] +
                    1;
            else
                L[i][j] = max(L[i-1][j],
                    L[i][j-1]);
        }
    }
    // Following code is used to print
        LCS
    int index = L[m][n];
    // Create a string length index+1 and
    // fill it with \0
    string lcs(index+1, '\0');
    // Start from the right-most-bottom-
        most
    // corner and one by one store
        characters
    // in lcs[]
    int i = m, j = n;
    while (i > 0 && j > 0){
        // If current character in X[]
            and Y are same, then current
            character is part of LCS
        if (X[i-1] == Y[j-1]) {
            // Put current character in
                result
            lcs[index-1] = X[i-1];
            i--; j--;
            // reduce values of i, j and
                index
            index--;
        }
        // If not same, then find the
            larger of
        // two and go in the direction of
            larger
        // value
        else if(L[i-1][j]>L[i][j-1])i--;
        else j--;
    }
    return lcs;
}
string longestPalSubseq(string &str) {
    string rev = str;
    reverse(rev.begin(), rev.end());
    return lcs(str, rev);
}
int main() {
    string str = "GEEKSFORGEEKS";
   cout << longestPalSubseq(str) <<endl;
      return 0;}
```

Mantenha Isso Energizado - Questão do Erick

```
#include<bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0);
#define MAXN 100020
#define i64 long long
#define OUT -5
#define vi vector<i64>
#define oo (1LL<<60)
using namespace std;
struct st{
   int L;
   i64 S, C;
}loja[MAXN];
int n, m;
vi custo, acum, ans;
vi tree;
bool cmp(st a, st b){
   return (a.L > b.L);
}
int bb(int idx){
   int lo = loja[idx].L, hi = n + 1, mid,
       r;
   i64 energia = loja[idx].S;
   if(energia < acum[loja[idx].L] - acum[
       loja[idx].L - 1]) return -1;
   while(lo < hi){
      mid = lo + (hi - lo) / 2;
      if(energia < (acum[mid] - acum[loja
          [idx].L - 1])) hi = mid;
      else lo = mid + 1;
   }
   return lo;
}
int left(int x){ return (x << 1); };
int right(int x){ return ((x << 1) + 1);
    };
int mid(int a, int b){ return ((a + b) >>
    1); };
i64 operation(i64 a, i64 b){
   return min(a, b);
}
i64 build(int u, int l, int r){
   if(l == r) return tree[u] = oo;
   build(left(u), l, mid(l, r));
   build(right(u), mid(l, r) + 1, r);
   return tree[u] = operation(tree[left(u
       )], tree[right(u)]);
}
i64 query(int u, int i, int j, int l, int
    r){
   if((r < i) || (l > j)) return OUT;
   if((l <= i) && (j <= r)) return tree[u
       ];
   i64 lans = query(left(u), i, mid(i, j)
       , l, r);
   i64 rans = query(right(u), mid(i, j) +
       1, j, l, r);
   if(lans == OUT) return rans;
   if(rans == OUT) return lans;
   return operation(lans, rans);
}
i64 update(int u, int i, int j, int index
    , i64 value){
   if((index < i) || (index > j)) return
       OUT;
   if(i == j) return tree[u] = value;
   update(left(u), i, mid(i, j), index,
       value);
   update(right(u), mid(i, j) + 1, j,
       index, value);
   return tree[u] = operation(tree[left(u
       )], tree[right(u)]);
}
int main(){_
   while(cin >> n >> m){
      custo = vi(n + 2);
      acum = vi(n + 2, 0LL);
      ans = vi(n + 2, oo); ans[n + 1] =
          0;
      tree = vi(4 * (n + 2));
      build(1, 1, n + 1);
      update(1, 1, n + 1, n + 1, 0);
      for(int i = 1; i <= n; ++i){
         cin >> custo[i];
         acum[i] = custo[i] + acum[i -
             1];
      }
      acum[n + 1] = acum[n];
      for(int i = 0; i < m; ++i){
         cin >> loja[i].L >> loja[i].S >>
              loja[i].C;
      }
      sort(loja, loja + m, cmp);
      int lim;
      for(int i = 0; i < m; ++i){
         lim = bb(i);
         if(lim != -1){
            ans[loja[i].L] = min(ans[loja
                [i].L], query(1, 1, n +
                1, loja[i].L, lim) + loja
                [i].C);
            update(1, 1, n + 1, loja[i].L
                , ans[loja[i].L]);
         }
      }
      if(ans[1] != oo) cout << ans[1] <<
          endl;
      else cout << "-1\n";
   }
   return 0;
}
```

Homem Elefante Rato

```
#include<bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0);
#define MAXN 100123
#define ll long long
#define vi vector<int>
#define vvi vector<vi>
#define left(x) (x << 1)
#define right(x) ((x << 1) + 1)
#define mid(a, b) ((a + b) >> 1)
using namespace std;
int size;
vi a, lazy, OUT;
vvi tree;
inline vi comp(vi va, vi vb){
   for(int i = 0; i < 3; ++i) va[i] += vb
       [i];
   return va;
}
inline void rotate(vi& va, int k){
   while(k--){
      swap(va[1], va[2]);
      swap(va[0], va[1]);
   }
}
inline void build(int u = 1, int i = 1,
    int j = size){
   if(i == j) {
      tree[u] = OUT;
      return;
   }
   build(left(u), i, mid(i, j));
   build(right(u), mid(i, j) + 1, j);
   tree[u] = comp(tree[left(u)], tree[
       right(u)]);
   return;
}
inline void push(int u, int i, int j){
   if(lazy[u] == 0) return;
   rotate(tree[u], lazy[u]);
   if(i != j){
      lazy[left(u)] += lazy[u];
      lazy[left(u)] %= 3;
      lazy[right(u)] += lazy[u];
      lazy[right(u)] %= 3;
   }
   lazy[u] = 0;
}
inline void update(int u, int i, int j,
    int l, int r){
   push(u, i, j);
   if((i > r) || (j < l)) return;
   if((l <= i) && (j <= r)){
      lazy[u]++;
      lazy[u] %= 3;
      push(u, i, j);
      return;
   }
   update(left(u), i, mid(i, j), l, r);
   update(right(u), mid(i, j) + 1, j, l,
       r);
   tree[u] = comp(tree[left(u)], tree[
       right(u)]);
   return;
}
inline vi query(int u, int i, int j, int
    l, int r){
   if((i > r) || (j < l)) return vi(3, 0)
       ;
   push(u, i, j);
   if((l <= i) && (j <= r)) return tree[u
       ];
   vi lans = query(left(u), i, mid(i, j),
        l, r);
   vi rans = query(right(u), mid(i, j) +
       1, j, l, r);
   return comp(lans, rans);
}
inline void update(int l, int r) { update
    (1, 1, size, l, r); }
inline vi query(int l, int r) { return
    query(1, 1, size, l, r); }
int main(){_
   int n, q;
   int l, r;
   char op;
   OUT = vi(3, 0);
   OUT[0] = 1;
   while(cin >> n >> q){
      size = n;
      a = vi(n + 1, 0);
      tree = vvi(4 * (n + 1), vi(3, 0));
      lazy = vi(4 * (n + 1), 0);
      build();
      vi ans;
      while(q--){
         cin >> op;
         if(op == 'M'){
            cin >> l >> r;
            update(min(l, r), max(l, r));
         }
         else if(op == 'C'){
            cin >> l >> r;
            ans = query(min(l, r), max(l,
                r));
            cout << ans[0] << ' ' << ans
                [1] << ' ' << ans[2] <<
                endl;
         }
      }
      cout << endl;
   }
   return 0;
}
```

Acordes Intergaláticos

```
#include<bits/stdc++.h>
#define _ ios_base::sync_with_stdio(0);
#define MAXN 100012
#define vi vector<int>
#define vvi vector<vi>
#define left(x) (x << 1)
#define right(x) ((x << 1) + 1)
#define mid(a, b) ((a + b) >> 1)
#define OUT 0

using namespace std;

int n;
vvi tree;
vi lazy, fora, final;

inline void rotate(vi &v, int k){
   while(k--){
      for(int w = 0; w < 8; ++w){
         swap(v[w], v[8]);
      }
   }
}

inline void build(int u, int i, int j){
   if(i == j) {
      tree[u][1] = 1;
      return;
   }

   build(left(u), i, mid(i, j));
   build(right(u), mid(i, j) + 1, j);

   for(int w = 0; w < 9; ++w){
      tree[u][w] = tree[left(u)][w] +
          tree[right(u)][w];
   }
}

inline void push(int u, int i, int j){
   if(lazy[u] == OUT) return;

   lazy[u] %= 9;
   rotate(tree[u], lazy[u]);

   if(i != j){
      lazy[left(u)] += lazy[u];
      lazy[right(u)] += lazy[u];
   }

   lazy[u] = OUT;
}

inline void update(int u, int i, int j,
    int l, int r, int value){
   push(u, i, j);

   if((i > r) || (j < l)) return;

   if((l <= i) && (j <= r)){
      lazy[u] += value;
      push(u, i, j);
      return;
   }

   update(left(u), i, mid(i, j), l, r,
       value);
   update(right(u), mid(i, j) + 1, j, l,
       r, value);

   for(int w = 0; w < 9; ++w){
      tree[u][w] = tree[left(u)][w] +
          tree[right(u)][w];
   }
}

inline vi query(int u, int i, int j, int
    l, int r){
   push(u, i, j);

   if((i > r) || (j < l)) return fora;

   if((l <= i) && (j <= r)) return tree[u
       ];

   vi lans = query(left(u), i, mid(i, j),
       l, r);
   vi rans = query(right(u), mid(i, j) +
       1, j, l, r);
   vi ans = fora;

   for(int w = 0; w < 9; ++w){
      ans[w] = lans[w] + rans[w];
      tree[u][w] = tree[left(u)][w] +
          tree[right(u)][w];
//    tree[u][w] = (lans[w] + rans[w]) %
    9; fazer s  isso e retornar tree[u]
    d  merda pois lans ou rans pode ser
    fora
   }
   return ans;
}

inline vi query(int l, int r){
   return query(1, 1, n, l, r);
}

inline void update(int l, int r, int
    value){
   update(1, 1, n, l, r, value);
}

inline void getAns(int u, int i, int j){
   push(u, i, j);
   if(i == j){
      final[i] = -1;
```

```
        for(int w = 0; w < 9; ++w) if(tree[
            u][w]) final[i] = w;
        return;
    }

    getAns(left(u), i, mid(i, j));
    getAns(right(u), mid(i, j) + 1, j);
}

int main(){
    fora = vi(9, 0);

    vi ans;
    int eleMaiorFreq, q, maiorFreq, l, r;

    scanf("%d %d", &n, &q);

    tree = vvi(4 * (n + 2), vi(9, 0));
    lazy = vi(4 * (n + 2), 0);
    final = vi(n + 1);

    build(1, 1, n);

    ans = query(1, n);

    while(q--){
        scanf("%d %d", &l, &r);
        ++l, ++r;
        ans = query(l, r);
        maiorFreq = *max_element(ans.begin
            (), ans.end());
        for(int i = 0; i < 9; ++i) if(
            maiorFreq == ans[i])
            eleMaiorFreq = i;

        ans = query(1, n);

        update(l, r, eleMaiorFreq);

    }
    getAns(1, 1, n);
    for(int j = 1; j <= n; ++j) printf("%d
        \n", final[j]); //cout << final[j]
         << endl;
    return 0;
}
```